

Deep Generative Models

Chapter 4: Generative Adversarial Networks

Ali Bereyhi

`ali.bereyhi@utoronto.ca`

Department of Electrical and Computer Engineering
University of Toronto

Summer 2025

Low-dimensional Latent

From our discussions on **latent space**: we expect that the **latent** be of **much lower dimension** $m \ll d$

! We could **not** work with such latent in **flow-based models**

↳ The **flow** needs to be **invertible**

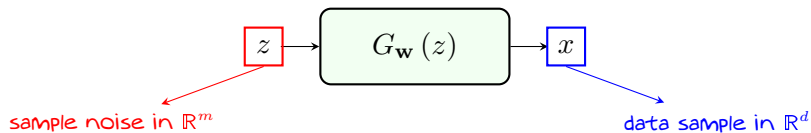
💡 We however **intuitively** expect that **a much smaller latent** could do the job

-
- + What if we try mapping a **low-dimensional latent** into the **data space**?
 - + Can't we use simply **MLE**?!
 - **Not explicitly!** Let's take a look

Generator: *Non-invertible Flow*

Generator Model

Generator model $G_{\mathbf{w}} : \mathbb{R}^m \mapsto \mathbb{R}^d$ is a mapping that maps a *latent sample* $z \sim Q(z)$, typically distributed *Gaussian*, into a data sample



- + Isn't that what we called *flow*?!
- Not exactly! *Flow* is *invertible*

Generator Distribution: *Example*

Consider a dummy example: $z \in \mathbb{R}$ and $x \in \mathbb{R}^3$ with *generator*

$$G(z) = [-z^2, \exp\{-z\}, z]$$

Say, want to find *the CDF* at $x = [x_1, x_2, x_3]$: we need to find

$$\begin{aligned} \mathbb{Z}(x) &= \{z \in \mathbb{R} : G(z) \leq x\} \\ &= \{z \in \mathbb{R} : -z^2 \leq x_1 \text{ and} \\ &\quad \exp\{-z\} \leq x_2 \text{ and} \\ &\quad z \leq x_3\} \end{aligned}$$

Generator Distribution: *Example*

① *First constraint requires*

$$-z^2 \leq x_1 \rightsquigarrow z^2 \geq -x_1 \rightsquigarrow \begin{cases} z \geq \sqrt{-x_1} \\ z \leq -\sqrt{-x_1} \end{cases}$$

② *Second constraint requires*

$$\exp\{-z\} \leq x_2 \rightsquigarrow -z \leq \log x_2 \rightsquigarrow z \geq -\log x_2$$

③ *And, finally the **third** one restricts z to **satisfy***

$$z \leq x_3$$

So, at $x = [x_1, x_2, x_3]$ we have

$$\mathbb{Z}(x) = \left([-\infty, -\sqrt{-x_1}] \cup [\sqrt{-x_1}, +\infty] \right) \cap [-\log x_2, \infty] \cap [-\infty, x_3]$$

Generator Distribution: *Example*

➤ At $x = [-1, \exp\{2\}, 2]$, we have

$$\mathbb{Z}(x) = [-2, -1] \cup [1, 2] \rightsquigarrow \text{CDF}(x) = \int_{-2}^{-1} P(z) dz + \int_1^2 P(z) dz$$

➤ At $x = [-1, 1, 2]$, we have

$$\mathbb{Z}(x) = [1, 2] \rightsquigarrow \text{CDF}(x) = \int_1^2 P(z) dz$$

➤ At $x = [a, 1, 2]$ with $a > 0$, we have

$$\mathbb{Z}(x) = \emptyset \rightsquigarrow \text{CDF}(x) = 0$$

Expressing $P(x)$ for *all* $x \in \mathbb{R}^3$ is quite *cumbersome!*

MLE Challenge: Non-Computability of Likelihood

Computability of Generator Distribution

With a **non-invertible** generator $G_{\mathbf{w}}$, it is challenging to characterize

$$\mathbb{Z}_{\mathbf{w}}(x) = \{z \in \mathbb{R}^m : G_{\mathbf{w}}(z) \leq x\}$$

and thus the output distribution $P_{\mathbf{w}}(x)$ is hard to be computed for all $x \in \mathbb{R}^d$

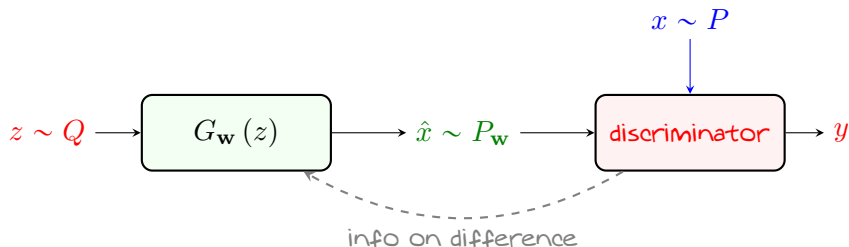
To train the **generator** by MLE $\rightsquigarrow$ we need **likelihood** at **all samples** x^j

$$\hat{R}(\mathbf{w}) = -\frac{1}{n} \sum_j \log P_{\mathbf{w}}(x^j)$$


Computability of Likelihood

With **non-invertible** generator, **likelihood** is **not** directly computable

Alternative Learning Approach: Adversarial Architecture



Discriminator

Discriminator is a **binary classifier** that classifies sample x as **real** or **fake**

To train with discriminator: we can **easily** make a **labeled** dataset

- **Data samples** are **labeled** as **real**
- **Generated samples** are **labeled** as **fake**

Dataset for Adversarial Architecture

From **discriminator** viewpoint: we train a **classifier** on $2n$ **labeled** samples

$$\mathbb{D} = \left\{ \boxed{(x^j, 1)}, \boxed{(\hat{x}^j, 0)} : j = 1, \dots, n \right\}$$

← real data sample → fake generated sample

We can compactly write the **dataset** as

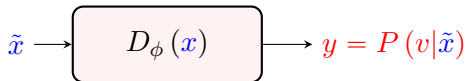
$$\mathbb{D} = \left\{ (\tilde{x}^j, v^j) : j = 1, \dots, 2n \right\}$$

where the **true label** v is defined as

$$v = \begin{cases} 0 & x \text{ if } \text{fake} \equiv \tilde{x} = G_{\mathbf{w}}(z) \\ 1 & x \text{ if } \text{real} \equiv \tilde{x} = x \sim P(x) \end{cases}$$

Training Computational Discriminator

We now consider a **computational discriminator**: a model $D_\phi : \mathbb{R}^d \mapsto [0, 1]$, e.g., NN with Sigmoid output, which classifies sample x



Recall

This is a **discriminative** model! This explains the **appellation** 😊

This is a classical **classification problem** $\rightsquigarrow$ we use **cross-entropy loss**

$$\begin{aligned}
 \text{CE}(y, v) &= -v \log y - (1 - v) \log (1 - y) \\
 &= \begin{cases} -\log y & \text{if } \tilde{x} \text{ is real} \equiv v = 1 \\ -\log (1 - y) & \text{if } \tilde{x} \text{ is fake} \equiv v = 0 \end{cases}
 \end{aligned}$$

Empirical Risk of Adversarial Architecture

To train on this **labeled dataset** $\rightsquigarrow$ we minimize the **empirical risk**, i.e.,

$$\begin{aligned}
 \hat{R}(\mathbf{w}, \phi) &= \frac{1}{2n} \sum_j \text{CE}(y^j, v^j) \\
 &= \frac{1}{2n} \sum_j \text{CE}(D_\phi(\tilde{x}^j), v^j) \\
 &= \frac{1}{n} \sum_j \text{CE}(D_\phi(x^j), 1) + \frac{1}{n} \sum_j \text{CE}(D_\phi(G_{\mathbf{w}}(z^j)), 0) \\
 &= \hat{\mathbb{E}}_{x \sim P} \{ \text{CE}(D_\phi(x), 1) \} + \hat{\mathbb{E}}_{z \sim Q} \{ \text{CE}(D_\phi(G_{\mathbf{w}}(z)), 0) \} \\
 &= \hat{\mathbb{E}}_{x \sim P} \{ -\log D_\phi(x) \} + \hat{\mathbb{E}}_{z \sim Q} \{ -\log(1 - D_\phi(G_{\mathbf{w}}(z))) \} \\
 &= - \left[\hat{\mathbb{E}}_{x \sim P} \{ \log D_\phi(x) \} + \hat{\mathbb{E}}_{z \sim Q} \{ \log(1 - D_\phi(G_{\mathbf{w}}(z))) \} \right]
 \end{aligned}$$

Empirical risk depends on both **generator** and **discriminator**

Training by Min-Max Game: *Discriminator's Role*

To train the **adversarial architecture** by the **empirical risk**

$$\hat{R}(\mathbf{w}, \phi) = - \left[\hat{\mathbb{E}}_{x \sim P} \{ \log D_{\phi}(x) \} + \hat{\mathbb{E}}_{z \sim Q} \{ \log (1 - D_{\phi}(G_{\mathbf{w}}(z))) \} \right]$$

we let the **discriminator** and **generator** play a **game**

Discriminator tries its best to **correctly classify fake** sample from **true ones**, i.e.,

$$\begin{aligned} \min_{\phi} \hat{R}(\mathbf{w}, \phi) &\equiv \max_{\phi} \left[\hat{\mathbb{E}}_{x \sim P} \{ \log D_{\phi}(x) \} + \hat{\mathbb{E}}_{z \sim Q} \{ \log (1 - D_{\phi}(G_{\mathbf{w}}(z))) \} \right] \\ &\equiv \max_{\phi} \mathcal{L}(\mathbf{w}, \phi) \end{aligned}$$


After maximization, **discriminator's best try** gets the objective

$$\mathcal{L}^*(\mathbf{w}) = \max_{\phi} \mathcal{L}(\mathbf{w}, \phi)$$

Training by Min-Max Game: Generator's Role

Generator tries its best to *confuse discriminator*

💡 This way *discriminator cannot* distinguish between *fake* and *true samples*

↳ This means that *fake samples* look the same as the *true ones*

💡 The *generator* can *only* play with its *parameters $\mathbf{w}$*

Generator tries to make *discriminator's best try as bad as possible*, i.e.,

$$\min_{\mathbf{w}} \mathcal{L}^{\star}(\mathbf{w}) = \min_{\mathbf{w}} \max_{\phi} \mathcal{L}(\mathbf{w}, \phi)$$

Statistical Perspective

Generator minimizes *discriminator's maximal likelihood* $\rightsquigarrow$ *discriminator remains confused* between *generated* and *true* samples even if it does *its best*

GAN: Generative Adversarial Network

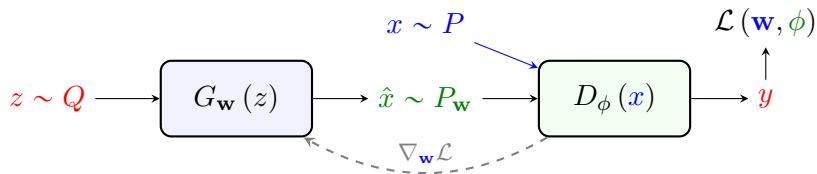
Generative Adversarial Network

GAN consists of a **generator** $G_{\mathbf{w}}$ and **discriminator** D_{ϕ} that are jointly learned through the **min-max game**

$$\min_{\mathbf{w}} \max_{\phi} \mathcal{L}(\mathbf{w}, \phi)$$

with objective function

$$\mathcal{L}(\mathbf{w}, \phi) = \hat{\mathbb{E}}_{x \sim P} \{\log D_{\phi}(x)\} + \hat{\mathbb{E}}_{z \sim Q} \{\log (1 - D_{\phi}(G_{\mathbf{w}}(z)))\}$$



GAN: Training Loop

Train_vanillaGAN($\mathbb{D}$:dataset):

```

1: Initiate the generator  $G_{\mathbf{w}}$  and discriminator  $D_{\phi}$  with some  $\mathbf{w}$  and  $\phi$ 
2: for multiple epochs do
3:   Sample a batch of data samples  $\{x^j : j = 1, \dots, n\}$  from  $\mathbb{D}$ 
4:   Sample latents  $\{z^j : j = 1, \dots, n\}$  using  $Q$  and compute  $\hat{x}^j \leftarrow G_{\mathbf{w}}(z^j)$ 
5:   for  $\xi = 1, \dots, \Xi$  do
6:     for  $j = 1, \dots, n$  do
7:       Compute  $\mathcal{L}^j = \log D_{\phi}(x^j) + \log(1 - D_{\phi}(\hat{x}^j))$ 
8:       Backpropagate over discriminator to compute  $\nabla_{\phi} \mathcal{L}^j$ 
9:       if  $\xi = \Xi$  then Backpropagate over generator to compute  $\nabla_{\mathbf{w}} \mathcal{L}^j$ 
10:    end for
11:    Update  $\phi$  using  $\text{Opt\_avg}\{-\nabla_{\phi} \mathcal{L}^j\}$ 
12:  end for
13:  Update  $\mathbf{w}$  using  $\text{Opt\_avg}\{\nabla_{\mathbf{w}} \mathcal{L}^j\}$ 
14: end for
15: return trained generator  $G_{\mathbf{w}}$ 

```

GAN Training: Few Notes

There are a few tricks to make training work

- 1 We update **discriminator** for Ξ steps **before** updating **generator once**

↳ This allows the **inner maximization** to slightly converge

$$\min_{\mathbf{w}} \max_{\phi} \mathcal{L}(\mathbf{w}, \phi)$$

↳ Typical choices are $5 \leq \Xi \leq 10$

- 2 To update **generator**, we only need to compute derivative of second term

$$\nabla_{\mathbf{w}} \mathcal{L}^j = \underbrace{\nabla_{\mathbf{w}} \log D_{\phi}(x^j)}_0 + \nabla_{\mathbf{w}} \log(1 - D_{\phi}(G_{\mathbf{w}}(z^j)))$$

↳ This gives **very small gradients first** and **exploding later**

↳ **First** $D_{\phi}(G_{\mathbf{w}}(z^j)) \approx 0$ and later when it's **fooled** $D_{\phi}(G_{\mathbf{w}}(z^j)) \approx 1$

↳ This is **opposite** of what we want!

↳ Heuristic remedy is to use $-\nabla_{\mathbf{w}} \log D_{\phi}(G_{\mathbf{w}}(z^j))$ instead

↳ Some people call $\mathcal{L}_G^j = -\log D_{\phi}(G_{\mathbf{w}}(z^j))$ **generator loss**

GAN: Sampling

Sampling is *exactly* as in *flow-based models*

```
Sample_GAN():
```

- 1: Sample latent as $z \sim Q(z)$ #Gaussian noise
- 2: Pass through generator $x \leftarrow G_w(z)$
- 3: return generated sample x

Though working with *tricks*, the *vanilla* GAN training is *generally unstable*

Wasserstein GAN addresses this issue by a very *smart trick*

To understand the *trick* used by *Wasserstein GAN*, we need to first learn the *statistical interpretation* of *vanilla* GAN training